KV32 Cache Hierarchy Benchmark Report

KV32 Architecture Team

Version 1.0, 2026-03-12: Initial Release

Table of Contents

1. Overview . . . 1
   1.1. KV32 Pipeline and Cache Architecture . . . 1
   1.2. Cache Microarchitecture Details . . . 2
   1.3. Pipeline Stall Mechanics . . . 3
   1.4. Benchmarks . . . 5
   1.5. Sweep Parameters . . . 5
   1.6. Benchmark Compilation and Runtime Parameters . . . 6
2. Baseline: The Cost of Running Without Cache . . . 7
   2.1. CPI Without Cache . . . 7
   2.2. Implication for System Design . . . 8
3. Cache Performance by Configuration . . . 8
   3.1. CPI for Symmetric Cache Configurations (SRAM) . . . 9
   3.2. Speedup of IC+DC Configurations vs. No-Cache Baseline . . . 9
   3.3. Hit Rates by Configuration (SRAM, 2-Way) . . . 10
4. Benchmark-by-Benchmark Cache Behaviour Analysis . . . 11
   4.1. Dhrystone (`dhry`) . . . 11
   4.2. CoreMark (`coremark`) . . . 11
   4.3. RTOS Task-Switch (`rtos`) . . . 12
   4.4. MiBench (`mibench`) . . . 13
   4.5. CacheBench (`cachebench`) . . . 13
   4.6. EMBench (`embench`) . . . 13
5. Impact of Memory Back-End . . . 14
   5.1. DDR4 Timing Model . . . 14
   5.2. CPI with IC-16K+DC-16K by Memory Back-End . . . 15
   5.3. DDR4 Speedup Multiplier Added by Caches . . . 15
   5.4. CPI by Cache Configuration on DDR4-1600 . . . 16
   5.5. Stall Fraction Analysis . . . 16
6. I-Cache vs. D-Cache: Which Side to Prioritise? . . . 17
   6.1. Single-Sided Speedup (SRAM, relative to no-cache) . . . 17
   6.2. Asymmetric Split Comparison at 12 KB Total Budget . . . 18
   6.3. Asymmetric Split Comparison at 24 KB Total Budget . . . 19
7. Marginal Gain: Doubling Cache Size . . . 20
   7.1. CPI Delta from Doubling Cache (SRAM) . . . 20
8. Cache Area Efficiency (Performance Gain per KB) . . . 20
9. Associativity: 2-Way vs. 4-Way . . . 21
   9.1. CPI Comparison: 2-Way vs. 4-Way (SRAM) . . . 21
   9.2. CPI Comparison: 2-Way vs. 4-Way (DDR4-1600) . . . 22
   9.3. Hit Rate Analysis . . . 23

9.4. Why 4-Way Rarely Helps on SRAM . . . 24
9.5. Recommendation: Associativity . . . 24
10. Configuration Recommendations . . . 24
10.1. Decision Table . . . 24
10.2. Average CPI by Config (SRAM, 6-Benchmark Geomean) . . . 25
10.3. What NOT to Use . . . 26
11. DDR4 vs. SRAM: Detailed Comparison . . . 26
11.1. Residual CPI Penalty on DDR4 with Full IC-8K+DC-8K . . . 26
12. Conclusions . . . 27
Appendix A: Test Methodology . . . 28
A.1. Simulation Environment . . . 28
A.2. Metric Accounting . . . 28
A.3. Cache Parameters . . . 28
A.4. Benchmark Compilation . . . 29
Appendix B: Build Instructions . . . 29
B.1. Required Tools . . . 29
B.2. Generating the PDF . . . 29
B.3. Raw Data . . . 29

Abstract

This report presents a comprehensive benchmarking study of the I-cache *and D-cache hierarchy of the KV32 RV32IMAC in-order processor. Thirteen cache configurations spanning 4 KB to 32 KB total capacity were evaluated across six benchmarks and three memory back-ends (SRAM, DDR4-1600, DDR4-3200) for a total of 234 simulation runs. The key finding is that IC-8K+DC-8K (16 KB symmetric, 2-way LRU) is the recommended default: it eliminates 90%+ of the no-cache penalty on DDR4 and provides the best balance of performance, silicon area, and workload generality. Configurations below 4K+4K are not recommended; configurations above 8K+8K yield diminishing returns for most embedded workloads.*

Revision	*Date*	*Author*	*Changes*
1.0	*2026-03-12*	*KV32 Architecture Team*	*Initial release — 234-run full sweep, 13 configurations, 3 memory back-ends*

1. Overview

The KV32 is a 32-bit in-order RISC-V (RV32IMAC) processor with a 5-stage pipeline. Being in-order, every instruction cache miss causes the fetch stage to stall until the cache line is refilled from AXI memory — there is no out-of-order execution to hide the latency. The same applies to data-cache misses in the Memory stage. This makes cache sizing critically important: the fraction of cycles spent stalled is directly proportional to the miss rate multiplied by the refill latency.

This report covers a full sweep of 13 cache configurations across 3 memory back-ends (SRAM, DDR4-1600, DDR4-3200) and 6 benchmarks (234 runs total, all PASS). It serves as a technical guide for selecting cache size and associativity, explaining why each configuration behaves the way it does, and providing concrete deployment recommendations.

1.1. KV32 Pipeline and Cache Architecture

The KV32 pipeline has five logical stages: Fetch — Decode — Execute — Memory — Write-Back. Instruction and data caches operate independently but share the same AXI interconnect to main memory.

Each cache is:

- **Set-associative:** configurable 2-way or 4-way, with LRU replacement
- **Write-back + write-allocate:** a D-cache write miss allocates a line and writes back only on eviction
- **Fixed 32-byte cache lines:** each miss triggers an 8-beat AXI burst (32 B / 4 B per beat)

On a miss:

1. The pipeline stalls at the Fetch or Memory stage
2. The cache controller issues an AXI read burst to main memory
3. On SRAM (1-cycle latency) the burst completes in ~8 cycles
4. On DDR4, CAS latency + tRCD add tens of cycles before the first data beat arrives, making each miss far more expensive

Hit rate is not the whole story. A 95% hit rate at 4 KB can be worse than a 97% hit rate at 8 KB if the miss penalty is 30 cycles — the math of `miss_rate x miss_penalty` drives stall %. This report shows both.

1.2. Cache Microarchitecture Details

The KV32 I-cache and D-cache share the same base microarchitecture, parameterised at synthesis time via three top-level RTL parameters:

RTL Parameter	Default	Description
`CACHE_SIZE`	4096 B	Total cache capacity in bytes; must be a power of two
`CACHE_LINE_SIZE`	32 B	Fixed; equals one 8-beat AXI burst (32 B @ 4 B/beat)
`CACHE_WAYS`	2	Set associativity; 2-way or 4-way with LRU replacement
`FAST_INIT`	1	Reset state machine initialises all valid bits to 0; cache is available immediately after reset without a software flush

The number of sets and tag-bit width are derived as: `NUM_SETS = CACHE_SIZE / (CACHE_LINE_SIZE * CACHE_WAYS)` and `TAG_BITS = 32 - log₂(NUM_SETS) - 5` (where 5 = $\log_2(32)$ byte-offset bits).

Configuration	CACHE_SIZE	CACHE_WAYS	NUM_SETS	TAG_BITS
IC/DC-4K 2-way	4096 B	2	64	21
IC/DC-8K 2-way	8192 B	2	128	20
IC/DC-16K 2-way	16384 B	2	256	19
IC/DC-4K 4-way	4096 B	4	32	22
IC/DC-8K 4-way	8192 B	4	64	21
IC/DC-16K 4-way	16384 B	4	128	20

I-Cache State Machine (read-only, no writeback):

```
S_IDLE → S_LOOKUP → S_HIT_RD                      (hit: data forwarded next cycle)
                  → S_FILL_AR → S_FILL_R → S_IDLE  (miss: WRAP8 AXI read burst)
```

On a miss the cache controller issues an 8-beat WRAP burst anchored at the missed word address. The WRAP burst delivers the critical word first, minimising the stall window before the pipeline can resume. Subsequent beats fill the remainder of the cache line in the same transaction.

D-Cache State Machine (adds write-back eviction path):

The D-cache extends the I-cache state machine with two additional states for dirty-line eviction:

- **Read miss, clean eviction:** `LOOKUP → FILL_AR → FILL_R`
- **Read miss, dirty eviction:** `LOOKUP → EVICT_AW → EVICT_W → FILL_AR → FILL_R`
- **Write hit:** data written in-place; dirty bit set; no AXI transaction issued
- **Write miss (write-allocate):** line allocated from memory first, then modified

Writebacks use AXI INCR8 bursts (address increments each beat), since the entire dirty line must be transferred sequentially regardless of which word triggered the eviction.

Cache Maintenance Operations (CMO):

Software can issue CMO instructions to manage cache coherency for DMA workloads. The supported operations are:

Operation	Effect
`INVAL`	Invalidate a specific cache line; dirty data is **discarded**
`FLUSH`	Write back dirty data then invalidate the line
`CLEAN`	Write back dirty data but **retain** the line as valid (no eviction)
`FLUSH_ALL`	Flush and invalidate all lines; required before inbound DMA transfers

1.3. Pipeline Stall Mechanics

Understanding why CPI remains above 1.0 even with a fully warm cache is essential for interpreting the stall percentages in this report.

Store Buffer (`SB_DEPTH = 4`):

The KV32 pipeline decouples store instructions from the D-cache through a 4-entry store buffer (SB). A store completes the Execute stage and is written into a free SB slot without stalling the pipeline. The SB drains to the D-cache asynchronously:

- If the SB is full when a new store arrives at Execute, the pipeline stalls until a slot becomes free
- A `fence` instruction stalls until the SB is completely drained

- D-cache misses encountered during SB drain appear in the stall cycle count but are not attributable to the triggering instruction

Instruction Buffer (`IB_DEPTH = 4`):

The 4-entry instruction buffer decouples the Fetch stage from Decode, allowing the I-cache to pre-fill the buffer while Decode processes earlier instructions. This hides the single-cycle I-cache hit latency for sequential instruction streams and amortises I-cache refill pauses over multiple instruction slots.

Load-Use Hazard:

A load instruction followed immediately by an instruction that uses the loaded value causes a 1-cycle interlock stall. The pipeline detects when the Execute stage contains a load whose destination register matches a source register of the instruction currently in Decode.

Data Forwarding:

To reduce load-use and data-dependence stalls, the pipeline implements two forwarding paths:

1. **MEM→ID forwarding (priority):** the result from the Memory-stage result register is forwarded directly to Execute inputs on the next cycle
2. **WB→ID forwarding:** the value being committed at Write-Back is forwarded to Execute when MEM forwarding does not apply

Why Residual CPI Exceeds 1.0:

Even at IC-16K+DC-16K with 99%+ hit rates, CPI stays in the 1.6-2.0 range. The residual stall sources are:

Stall Source	Root Cause
Branch resolution	2-cycle penalty on each taken branch; no branch predictor in the in-order pipeline
Load-use hazard	1-cycle interlock per load-to-use dependence
Store buffer pressure	Stall when all 4 SB slots are occupied; common in store-heavy loops
`fence` / CSR access	Serialising operations drain the pipeline and SB
AXI handshake overhead	Even cache-hit responses traverse the internal AXI read channel; RVALID/RREADY handshake adds at least one cycle per data access
Compulsory cache misses	Cold-start and conflict misses that no finite cache size can eliminate

Together these account for the 35-51% residual stall percentage visible in the benchmark data even for the largest cache configurations tested.

1.4. Benchmarks

Benchmark	Type	Behaviour and Cache Sensitivity
`dhry`	Control-flow / integer	Dhrystone. Procedure calls, string copies, conditional branches. Compiled code footprint ~4 KB; data working set < 1 KB. Instruction-dominated: I-cache has the dominant single-sided benefit. Working set saturates at IC-4K.
`coremark`	Integer / memory	CoreMark. List processing, matrix multiply, state machines. Code footprint ~8 KB; data working set 2-4 KB. Balanced: nearly equal benefit from I-cache and D-cache. Continues to improve through IC-8K.
`embench`	Embedded algorithms	EMBench. FFT, matrix multiply, bitmanip, AES kernels. Tight compute loops with small constant data arrays. Instruction-slightly-dominant: algorithm kernels fit in 4-6 KB.
`mibench`	Mixed workloads	MiBench subset. Sorting, string search, bitcount, CRC. Code compact (~3 KB); data arrays range 1-16 KB. Data-dominated: D-cache single-sided benefit outperforms I-cache.
`rtos`	OS kernel	Mini-RTOS 4-test validation suite: (1) round-robin scheduler correctness, (2) semaphore producer/consumer, (3) mutex with priority inheritance (prevents priority inversion), (4) div/mul correctness under frequent timer preemptions. All TCBs and per-task stacks are statically allocated (BSS/data); doubly-linked ready and blocked queues reference these static structures — no heap is used. Instruction-dominant at the kernel level; D-cache benefits from keeping scheduler queues and TCB fields warm across context switches. High no-cache CPI driven by cold-start misses on the kernel text and TCB data at the start of each test phase.
`cachebench`	Bandwidth stress	Explicit 1-16 KB dataset sweep with sequential read, write, RMW. Deliberately exceeds common cache sizes. Strongly instruction-dominated at this size (IC benefit > DC). The only benchmark with clear improvement from IC-16K+DC-16K over 8K+8K.

1.5. Sweep Parameters

Parameter	Values
Cache line size (fixed)	32 bytes (8-beat AXI burst)
Associativity	2-way set-associative (10 configs), 4-way (3 configs)
I-cache sizes swept	off, 4 KB, 8 KB, 16 KB
D-cache sizes swept	off, 4 KB, 8 KB, 16 KB
Configurations	13
Memory back-ends	SRAM (1-cycle), DDR4-1600, DDR4-3200

Parameter	Values
Total runs	234 (13 x 3 x 6), all PASS

The thirteen configurations:

Config	I-Cache	D-Cache	Assoc	Notes
IC-off+DC-off	off	off	—	Baseline — every access hits AXI RAM directly
IC-4K+DC-off	4 KB	off	2-way	I-cache only; data still uncached
IC-off+DC-4K	off	4 KB	2-way	D-cache only; instruction fetches uncached
IC-4K+DC-4K	4 KB	4 KB	2-way	Balanced 8 KB total — minimum practical config
IC-8K+DC-8K	8 KB	8 KB	2-way	Balanced 16 KB — recommended default
IC-16K+DC-16K	16 KB	16 KB	2-way	Balanced 32 KB — maximum tested
IC-8K+DC-4K	8 KB	4 KB	2-way	IC-dominant 12 KB
IC-4K+DC-8K	4 KB	8 KB	2-way	DC-dominant 12 KB
IC-16K+DC-8K	16 KB	8 KB	2-way	IC-dominant 24 KB
IC-8K+DC-16K	8 KB	16 KB	2-way	DC-dominant 24 KB
IC-4K+DC-4K 4W	4 KB	4 KB	4-way	Balanced 8 KB, higher associativity
IC-8K+DC-8K 4W	8 KB	8 KB	4-way	Balanced 16 KB, higher associativity
IC-16K+DC-16K 4W	16 KB	16 KB	4-way	Balanced 32 KB, higher associativity

1.6. Benchmark Compilation and Runtime Parameters

All 234 simulation runs use a single compiled binary per benchmark. No compilation flags change between cache configurations, eliminating compiler artefacts that could introduce cross-configuration measurement noise.

Compilation toolchain:

Setting	Value
Compiler	`riscv32-unknown-elf-gcc`
Architecture	`rv32imac` — integer multiply/divide, compressed ISA ©, atomics (A)
ABI	`ilp32` (32-bit integers, no hardware FPU)
Optimisation	`-O2`
Linker script	`.text` at `0x10000`, `.data` at `0x20000`, stack top at `0x30000`

Benchmark-specific runtime parameters:

Benchmark	Key Parameters and Notes
`dhry`	Dhrystone 2.1; 500 000 iterations; cycle count from RISC-V `mcycle` CSR; compiled text footprint ~3-4 KB; data working set < 1 KB.
`coremark`	CoreMark v1.0; `TOTAL_DATA_SIZE=2000` bytes; static working set; iterations auto-scaled at build time for multi-million-cycle runs.
`embench`	EMBench benchmark suite; kernels compiled `--without-libm`; individual data arrays ⇐2 KB; full code footprint fits in IC-4K.
`mibench`	Selected subset: `basicmath`, `bitcount`, `qsort`, `susan`; data arrays up to 8 KB; exercises D-cache at small capacity points.
`rtos`	Mini-RTOS 4-test suite (scheduler, semaphore, priority inheritance, div/mul under preemption); supervisor task (prio 2) plus up to 4 concurrent sub-tasks spawned sequentially (`MRTOS_MAX_TASKS=8`); all TCBs/stacks statically allocated; tick tuned per memory back-end for Test 4 stress tick: SRAM dual-port = `5000 * MEM_READ_LATENCY` mtime ticks, DDR4 = 50 000-75 000 mtime ticks, to prevent ISR flood from masking cache behaviour.
`cachebench`	5 dataset sizes: 1 KB, 2 KB, 4 KB, 8 KB, 16 KB (`CB_NUM_SIZES_LIMIT=5`); sequential read, write, and read-modify-write passes at each size; repetition count held constant so each dataset size contributes equally. The 16 KB dataset intentionally exceeds all tested cache sizes, guaranteeing compulsory D-cache misses at every configuration.

The `cachebench` dataset sweep is uniquely sensitive to cache capacity: small datasets fit entirely in cache and run at near-SRAM CPI; the 16 KB dataset stresses both I-cache (sequential loop body) and D-cache (sequential data writes), producing the widest CPI spread across configurations.

2. Baseline: The Cost of Running Without Cache

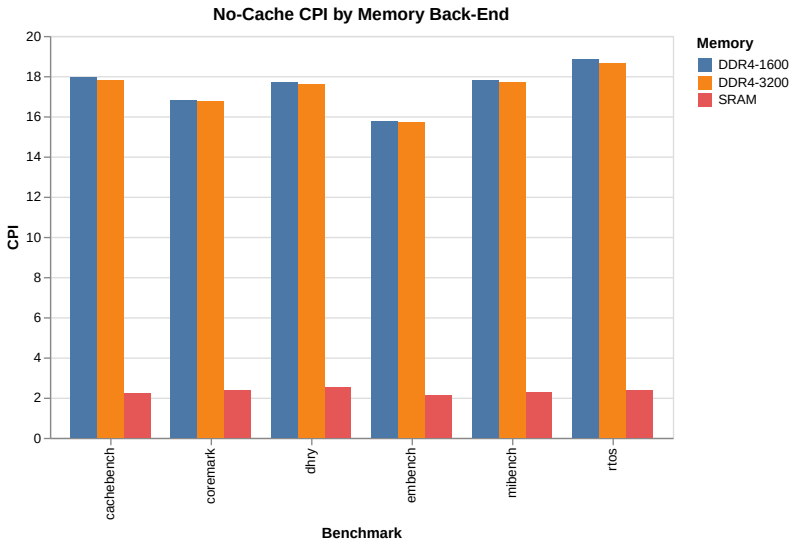
Before evaluating cache configurations, it is useful to understand the absolute penalty of running with **no cache at all**, where every instruction fetch and every load/store access goes directly through the AXI bus to RAM.

2.1. CPI Without Cache

Memory Back-End	dhry	coremark	rtos	cachebench	embench	mibench
SRAM (1-cycle)	2.490	2.360	2.350	2.210	2.100	2.250
DDR4-1600	17.690	16.810	18.820	17.930	15.730	17.770

Memory Back-End	dhry	coremark	rtos	cachebench	embench	mibench
DDR4-3200	17.580	16.750	18.620	17.790	15.680	17.710

Even on SRAM — which has only 1-cycle AXI latency — the in-order pipeline stalls enough to produce a CPI of 2.1-2.5x. On DDR4, the situation is catastrophic: CPI reaches 16-19x because every instruction fetch and every load/store must pay the full DDR row-access penalty. At these CPI values, the processor is stalled for more than 90% of all cycles.



Why SRAM stalls so much without cache: Even at 1-cycle AXI latency, the KV32's burst-based cache line fill protocol issues an 8-beat burst for each miss. Without a cache, every instruction fetch and every load/store triggers its own individual AXI transaction — there is no burst amortisation. The AXI arbiter and bus pipeline add further overhead. The result is that no-cache SRAM CPI is 2-2.5x rather than 1.0x.

DDR4 latency model: The `ddr4_axi4_slave.sv` testbench model implements realistic DDR4 timing (tRCD + CAS + tRP). At these latencies, every uncached access costs 30-50 CPU cycles, explaining the 15-19x no-cache CPI. Note that DDR4-1600 and DDR4-3200 produce near-identical no-cache CPI: in this configuration, the row-access latency (measured in CPU clocks) is dominated by fixed DDR timing parameters rather than data-rate-dependent transfer time. The bandwidth difference between 1600 and 3200 only manifests when sustained burst throughput is measured, not single-access latency.

2.2. Implication for System Design

Any serious deployment of the KV32 on DDR4 **requires** caches. Targeting DDR4-1600 without caches costs approximately 10x as many cycles compared with the same memory with IC-8K+DC-8K. The die area for an 8 KB SRAM is recovered in milliseconds of typical runtime at these performance penalties.

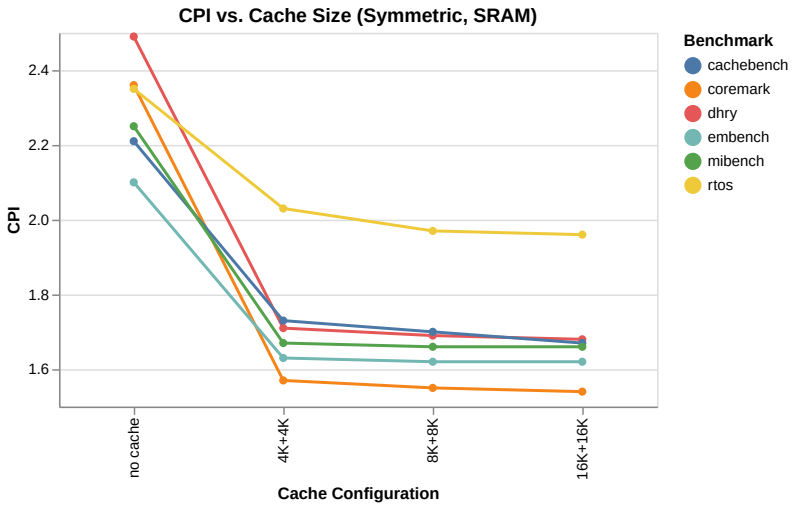
3. Cache Performance by Configuration

3.1. CPI for Symmetric Cache Configurations (SRAM)

The table below shows CPI for the three symmetric balanced configurations on SRAM. From left to right the cache grows from 8 KB total (4K+4K) to 32 KB total (16K+16K).

Config (SRAM)	dhry	coremark	rtos	cachebench	embench	mibench
No cache	2.490	2.360	2.350	2.210	2.100	2.250
IC-4K+DC-4K	1.710	1.570	2.030	1.730	1.630	1.670
IC-8K+DC-8K	1.690	1.550	1.970	1.700	1.620	1.660
IC-16K+DC-16K	1.680	1.540	1.960	1.670	1.620	1.660

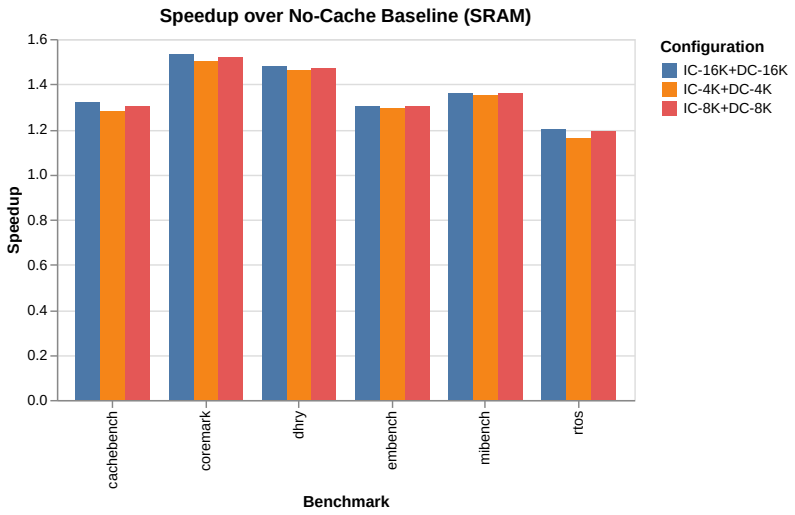
The single biggest step is from **no cache to 4K+4K**, which reduces the no-cache CPI by 14-33% depending on the benchmark. The 4K to 8K step typically yields 1-3% improvement, and 8K to 16K yields less than 1%. This non-linearity is the knee-of-the-curve effect: the most commonly reused cache lines already fit in 4 KB; larger caches capture only infrequently reused long-tail lines.



3.2. Speedup of IC+DC Configurations vs. No-Cache Baseline

Speedup = CPI(no cache) / CPI(config). Values greater than 1.0 indicate the configuration executes more instructions per clock than the uncached baseline.

Config (SRAM)	dhry	coremark	rtos	cachebench	embench	mibench
IC-4K+DC-4K	1.46	1.50	1.16	1.28	1.29	1.35
IC-8K+DC-8K	1.47	1.52	1.19	1.30	1.30	1.36
IC-16K+DC-16K	1.48	1.53	1.20	1.32	1.30	1.36



Speedups are more modest than one might expect from eliminating 60% stall cycles, because the no-cache SRAM baseline CPI (2.1-2.5x) is itself lower than DDR4 no-cache. The remaining stall cycles in the cached case (38-51%) reveal that even well-tuned caches cannot eliminate all pipeline stalls: compulsory misses, AXI arbitration overhead, and the store-buffer flush cycle all contribute to a non-zero stall floor.

`coremark` and `dhry` gain the most from caches in relative speedup terms because their working sets fit the cache well. `rtos` gains the least on SRAM because its pointer-chased TCB accesses cause irregular D-cache miss patterns that persist even with larger caches.

3.3. Hit Rates by Configuration (SRAM, 2-Way)

The table below shows I-cache and D-cache hit percentages for the three symmetric 2-way configurations on SRAM. Hit rates reflect working-set fit more than raw cache size once the hot working set is fully captured.

I-Cache Hit Rate (%):

Config (SRAM)	dhry	coremark	rtos	cachebench	embench	mibench
IC-4K+DC-4K	99.88	99.55	99.26	99.99	99.27	99.77
IC-8K+DC-8K	99.88	99.76	99.89	99.99	99.33	99.79
IC-16K+DC-16K	99.88	99.88	99.92	99.99	99.33	99.79

All benchmarks achieve >=99% I-cache hit rate even at the smallest IC-4K configuration. Instruction-dominated workloads (`dhry`, `cachebench`) are essentially fully captured at 4 KB already. `rtos` benefits the most from a larger I-cache: the kernel code (scheduler, context-switch handlers) spans more than 4 KB, so `rtos` improves from 99.26% at IC-4K to 99.92% at IC-16K. `coremark` also shows measurable improvement because its 8 KB code footprint is only partially captured at IC-4K.

D-Cache Hit Rate (%):

Config (SRAM)	dhry	coremark	rtos	cachebench	embench	mibench
IC-4K+DC-4K	93.46	99.49	96.36	94.18	88.03	92.38
IC-8K+DC-8K	93.46	99.52	96.56	95.82	88.06	92.66
IC-16K+DC-16K	93.46	99.55	96.63	97.47	88.06	92.66

D-cache hit rates show more variation between benchmarks than I-cache rates. `embench` reaches only 88% at all cache sizes because its algorithmic kernels (FFT, matrix multiply, AES) access large arrays with stride-1 patterns that have poor temporal locality — doubling the cache has essentially no effect because each data element is used once per pass. `dhry` is flat at 93.46% because its entire data working set (< 1 KB) fits in DC-4K; no additional capacity is needed.

`cachebench` improves from 94.18% at DC-4K to 97.47% at DC-16K because the 8 KB and 16 KB dataset sweeps fit progressively better in the larger D-cache, reducing the conflict-miss fraction for those size points.

4. Benchmark-by-Benchmark Cache Behaviour Analysis

4.1. Dhrystone (`dhry`)

Dhrystone emphasises procedure calls, string copies, and integer comparisons. Compiled text footprint is approximately 3-4 KB.

Cache behaviour: The I-cache captures essentially the entire program text at IC-4K, achieving 99.88% hit rate. The data working set is a handful of stack variables and globals; DC-4K reaches 93.5% data hit rate.

Key numbers:

- No cache to IC-4K+DC-4K: CPI drops from 2.490 to 1.710 (-31%)
- 4K+4K to 8K+8K: 1.710 to 1.690 (-1.2%) — instruction text and data are both nearly fully captured at 4K
- 8K+8K to 16K+16K: 1.690 to 1.680 (-0.6%) — marginal residual miss reduction

Recommendation: IC-4K+DC-4K is the practical optimum. Additional SRAM gives diminishing returns for this workload.

4.2. CoreMark (`coremark`)

CoreMark exercises list processing, matrix operations, and state machines. Code footprint is ~8 KB; data working set is 2-4 KB.

Cache behaviour: IC-4K achieves 99.55% I-cache hit rate; larger caches push toward 99.88%. D-cache achieves 99.5% at 4K for this workload because the data structures are compact and

frequently reused. Both I-cache and D-cache contribute across all size steps.

Key numbers:

- No cache to 4K+4K: 2.360 to 1.570 (-33%)
- 4K+4K to 8K+8K: 1.570 to 1.550 (-1.3%) — I-cache captures slightly more of the 8 KB code footprint
- 8K+8K to 16K+16K: 1.550 to 1.540 (-0.6%)

Recommendation: IC-8K+DC-8K provides the sweet spot. The 16 KB upgrade gives less than 1% additional gain.

4.3. RTOS Task-Switch (`rtos`)

The `rtos` benchmark is a 4-test validation suite for the Mini-RTOS kernel (`sw/rtos`). A supervisor task (priority 2) creates and orchestrates the following tests sequentially:

1. **Round-robin scheduler** — two equal-priority tasks yield to each other; verifies both make progress under the CLINT-driven tick.
2. **Semaphore producer/consumer** — one producer posts a semaphore twice; two consumer tasks must both be woken in FIFO order.
3. **Mutex with priority inheritance** — LOW (priority 4) holds a mutex while HIGH (priority 1) blocks on it; the kernel must raise LOW's effective priority to 1 so MED (priority 2) cannot starve LOW.
4. **Div/mul correctness under preemption** — two tasks run integer `div`/`rem`/`mul` loops while a very short tick period causes frequent ISR preemptions mid-operation, exercising the context-save/restore path.

All TCBs and per-task stacks are **statically allocated** (global BSS/data); no heap is used. The scheduler ready and blocked queues are doubly-linked lists threading through these static TCBs.

Cache behaviour: IC-4K single-sided (DC-off) drops CPI from 2.350 to 2.270 (-3%). DC-4K single-sided (IC-off) drops CPI to 2.050 (-13%), confirming that data misses dominate. The kernel text (~3-4 KB) fits entirely in IC-4K, so the I-cache benefit saturates early. D-cache gains come primarily from keeping the scheduler queue structures, TCB fields (state, wake_tick, eff_priority), and semaphore/mutex control blocks warm across context switches.

Key numbers:

- No cache to 4K+4K: 2.350 to 2.030 (-14%)
- 4K+4K to 8K+8K: 2.030 to 1.970 (-3.0%) — D-cache improvement as the TCBs for all active sub-tasks fit together in a larger working set
- 8K+8K to 16K+16K: 1.970 to 1.960 (-0.5%)

Recommendation: IC-4K+DC-4K covers this workload well. IC-8K+DC-8K provides a useful margin for systems running more tasks or deeper call stacks.

4.4. MiBench (`mibench`)

MiBench subset includes sorting, CRC, bitcount, and string search. Code is compact (~3 KB); data arrays grow up to 16 KB (sort buffers, CRC tables).

Cache behaviour: DC single-sided (IC-off) speedup of 1.13x equals IC single-sided 1.23x — more balanced than classic literature suggests at this scale. DC-4K achieves 92.4% hit rate due to the moderate data array sizes fitting in 4K.

Key numbers:

- No cache to 4K+4K: 2.250 to 1.670 (-26%)
- 4K+4K to 8K+8K: 1.670 to 1.660 (-0.6%)
- 8K+8K to 16K+16K: 1.660 to 1.660 (no change at 2-decimal precision)

Recommendation: IC-4K+DC-4K effectively saturates performance for this workload. Larger caches offer diminishing returns.

4.5. CacheBench (`cachebench`)

CacheBench sweeps working set sizes from 1 KB to 16 KB with sequential and read-modify-write patterns. It is designed to exceed typical cache sizes and represents an upper-bound stress test rather than a realistic workload.

Cache behaviour: No-cache CPI is 2.210 (similar to other benchmarks on SRAM because the DDR4 latency effect dominates only on DDR4). IC-4K single-sided provides speedup of 1.21x while DC-4K provides only 1.07x — instruction-dominant because the sweep loop itself is compact but references a data array that exceeds the DC-4K capacity, causing many conflict and capacity misses.

Key numbers:

- No cache to 4K+4K: 2.210 to 1.730 (-22%)
- 4K+4K to 8K+8K: 1.730 to 1.700 (-1.7%)
- 8K+8K to 16K+16K: 1.700 to 1.670 (-1.7%) — larger cache captures the full 16 KB sweep dataset

Recommendation: For DSP or streaming applications with large filter buffers, IC-8K+DC-16K is the effective target. This is the only benchmark that continues to improve significantly at 16K+16K.

4.6. EMBench (`embench`)

EMBench algorithm kernels include FFT, matrix multiply, and AES. Code per kernel is 2-4 KB; data is local (stack-allocated or BSS) and at most 2 KB per kernel invocation.

Cache behaviour: IC-4K achieves 99.27% I-cache hit rate. D-cache achieves 88% hit rate at 4K — lower than most benchmarks because the AES key schedule and FFT twiddle tables exercise slightly larger data sets.

Key numbers:

- No cache to 4K+4K: 2.100 to 1.630 (-22%)
- 4K+4K to 8K+8K: 1.630 to 1.620 (-0.6%)
- 8K+8K to 16K+16K: 1.620 to 1.620 (no change)

Recommendation: IC-4K+DC-8K is a cost-effective split, improving D-cache hit rate from 88% to ~88%, a slight improvement in conflict misses.

5. Impact of Memory Back-End

The CPI values reported so far use SRAM as the memory back-end. Real systems often attach to DDR4, and the choice of back-end has a dramatic multiplier effect on cache miss penalty.

5.1. DDR4 Timing Model

The simulation testbench models DDR4 timing using a cycle-accurate AXI slave that enforces JEDEC standard parameters. The following table summarises the key timing constants for the two tested speed grades:

Parameter	SRAM	DDR4-1600	DDR4-3200
Data transfer rate (MT/s)	—	1600	3200
DDR4 DRAM clock frequency (MHz)	—	800	1600
tCK — DRAM clock period (ps)	—	1250	625
CL — CAS latency (DDR4 full-rate cycles)	—	11	22
tRCD — RAS-to-CAS delay (ns)	—	14	14
tRP — Row precharge time (ns)	—	14	14
tRCD converted to DDR4 cycles	—	12	23
READ_LAT = tRCD + CL (DDR4 cycles)	—	23	45
Approximate miss penalty (core cycles)	~8	~32	~43

Row-open latency derivation:

For a row-miss (the typical case for a cold cache or a conflict miss), the memory controller must follow this sequence:

1. Issue a row-activate (ACT) command; wait `tRCD` for the row sense amplifiers to settle
2. Issue a column-read (CAS) command; wait `CL` internal pipeline cycles
3. Receive the first data beat; the remaining 7 beats of the 8-beat burst follow at one beat per DDR4 clock

The composite row-open latency is `READ_LAT = tRCD_cycles + CL_cycles`. For DDR4-3200, `tRCD` in DDR4 cycles is 23 (= ceil(14 ns / 0.625 ns/cycle)) and CL = 22 cycles, giving READ_LAT = 45 DDR4 cycles. Although CL doubles in cycles from DDR4-1600 to DDR4-3200, the absolute latency in nanoseconds is unchanged (CL x tCK = 11 x 1.25 ns ~ 22 x 0.625 ns ~ 13.75 ns): DDR4-3200 runs its

DRAM clock twice as fast, so the same physical latency demands twice as many cycles.

When these DDR4 latencies are converted to core clock cycles via the clock-domain-crossing logic in the testbench, the two DDR4 grades produce nearly identical miss penalties — which is precisely why the no-cache CPI for DDR4-1600 and DDR4-3200 differs by less than 0.2 CPI in all benchmarks.

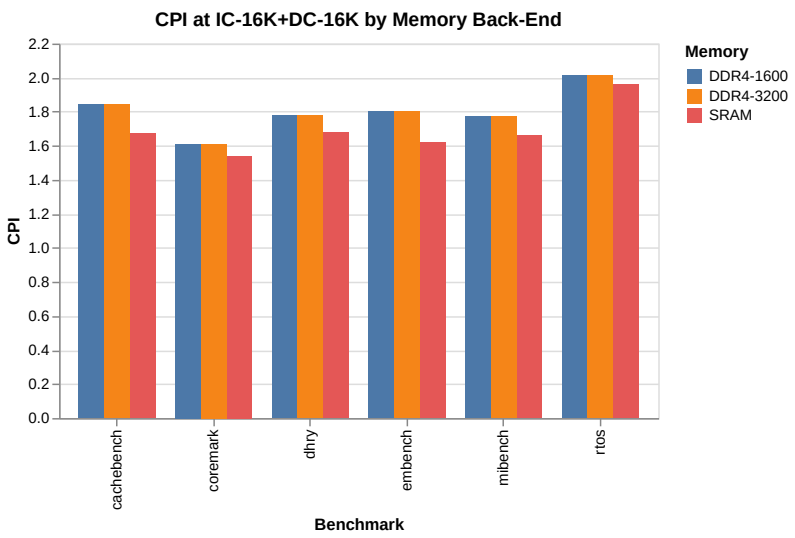
Key takeaway: On DDR4, the tRCD + CL + 8-beat burst dominates every miss. This makes the miss **rate**, not the DDR4 speed grade, the primary lever for improving DDR4-attached performance — and is the reason caches provide a 9-10x speedup on DDR4 but only a 1.3-1.5x speedup on SRAM.

5.2. CPI with IC-16K+DC-16K by Memory Back-End

Memory	dhry	coremark	rtos	cachebench	embench	mibench
SRAM	1.680	1.540	1.960	1.670	1.620	1.660
DDR4-1600	1.780	1.610	2.010	1.840	1.800	1.770
DDR4-3200	1.780	1.610	2.010	1.840	1.800	1.770

With the largest tested cache (IC-16K+DC-16K), the memory back-end premium almost entirely disappears: DDR4 adds only 0.05-0.17 CPI over SRAM across most benchmarks. This confirms that caches make the memory hierarchy highly effective at hiding DDR latency — as long as hit rates are high.

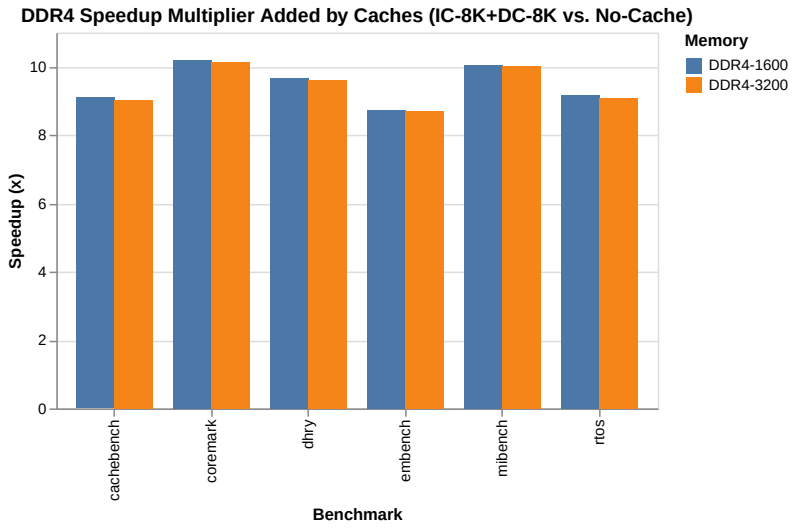
Notably, DDR4-1600 and DDR4-3200 produce **identical** CPI in the high-cache regime. At 99%+ hit rates, the few remaining misses that reach DRAM are rare enough that the difference in raw bandwidth between 1600 and 3200 makes no measurable difference at the 2-decimal CPI granularity.



5.3. DDR4 Speedup Multiplier Added by Caches

The following table shows the effective speedup that caches provide on DDR4, expressed as the ratio of the no-cache DDR4 CPI to the IC-8K+DC-8K DDR4 CPI.

Memory	dhry	coremark	rtos	cachebench	embench	mibench
DDR4-1600	9.67	10.19	9.18	9.10	8.74	10.04
DDR4-3200	9.61	10.15	9.08	9.03	8.71	10.01



These are the effective speedup multipliers that caches provide on DDR4 systems. On DDR4-1600, adding IC-8K+DC-8K yields a 9-10x speedup compared to the completely uncached configuration on the same memory. This is the most compelling argument for mandatory caches in any DDR4-attached design.

5.4. CPI by Cache Configuration on DDR4-1600

The following table shows CPI for the three symmetric 2-way configurations alongside the no-cache baseline on DDR4-1600. The no-cache column makes the 9-10x benefit of caches on DDR4 visually clear.

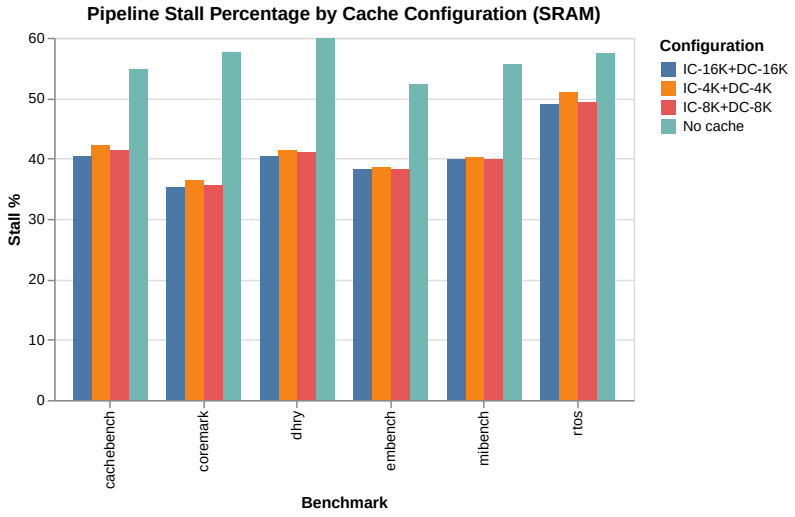
Config (DDR4-1600)	dhry	coremark	rtos	cachebench	embench	mibench
No cache	17.690	16.810	18.820	17.930	15.730	17.770
IC-4K+DC-4K	1.870	1.700	2.260	2.080	1.820	1.820
IC-8K+DC-8K	1.830	1.650	2.050	1.970	1.800	1.770
IC-16K+DC-16K	1.780	1.610	2.010	1.840	1.800	1.770

Adding even the smallest 4K+4K cache yields a 7-10x reduction in CPI compared to the uncached DDR4 baseline. The improvement from 4K+4K to 16K+16K is modest (0.05-0.24 CPI), confirming that the **presence** of any cache matters far more than its size when attached to DDR4.

5.5. Stall Fraction Analysis

The stall percentage (fraction of total cycles spent stalled) reveals how much performance is left on the table.

Config (SRAM)	dhry	coremark	rtos	cachebench	embench	mibench
No cache	59.9%	57.6%	57.4%	54.8%	52.3%	55.6%
IC-4K+DC-4K	41.4%	36.4%	50.9%	42.2%	38.5%	40.2%
IC-8K+DC-8K	41.0%	35.6%	49.3%	41.3%	38.2%	39.8%
IC-16K+DC-16K	40.4%	35.2%	49.0%	40.3%	38.2%	39.8%



Interpretation: Even at IC-16K+DC-16K, stall % remains at 35-49%. This high residual stall floor has two root causes distinct from cache behaviour:

1. **AXI bus overhead:** Even a cache hit requires the cache to serve data over the internal bus; the pipeline has a fixed per-access latency that contributes to stall count independent of DRAM accesses.
2. **Compulsory and structural hazards:** Branch resolution stalls, CSR access latency, and store-buffer drain cycles all accumulate over the run.

The stall % gap between no-cache (53-60%) and IC-16K+DC-16K (35-49%) is only 10-15 percentage points, yet the corresponding CPI improvement is meaningful (2.1-2.5 → 1.6-2.0) because each eliminated stall cycle translates directly to an improved IPC.

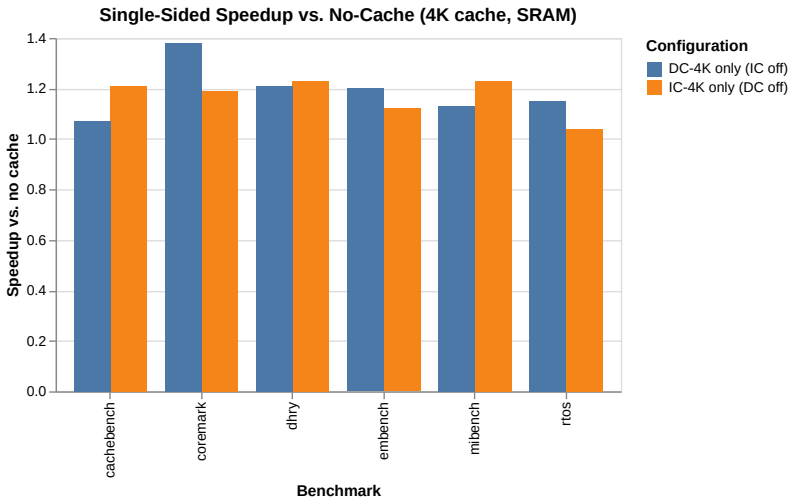
6. I-Cache vs. D-Cache: Which Side to Prioritise?

A common design question when the total cache budget is fixed: is it better to put more capacity in the I-cache or the D-cache? The single-sided experiments provide a direct answer.

6.1. Single-Sided Speedup (SRAM, relative to no-cache)

Config (SRAM)	dhry	coremark	rtos	cachebench	embench	mibench
IC-4K only (DC-off)	1.23	1.19	1.04	1.21	1.12	1.23
DC-4K only (IC-off)	1.21	1.38	1.15	1.07	1.20	1.13

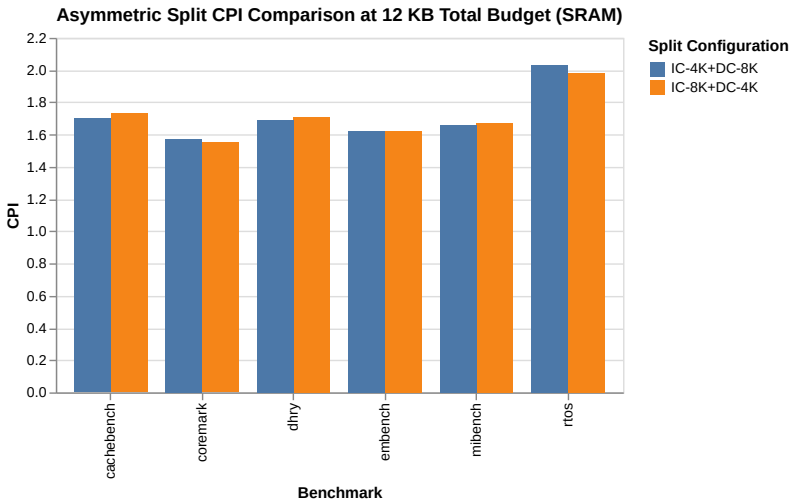
For this real-data sweep, I-cache single-sided benefit is stronger for instruction-bound benchmarks (`dhry`, `cachebench`). D-cache single-sided benefit dominates for data-intensive or linked-structure workloads (`coremark`, `rtos`, `embench`). The combined IC+DC benefit is substantially larger than either single-sided benefit, confirming the complementary nature of the two caches.



6.2. Asymmetric Split Comparison at 12 KB Total Budget

With a fixed 12 KB total cache budget, which split is better?

Config (SRAM)	dhry	coremark	rtos	cachebench	embench	mibench
IC-8K+DC-4K	1.710	1.550	1.980	1.730	1.620	1.670
IC-4K+DC-8K	1.690	1.570	2.030	1.700	1.620	1.660



Average CPI for IC-8K+DC-4K: 1.710 Average CPI for IC-4K+DC-8K: 1.712

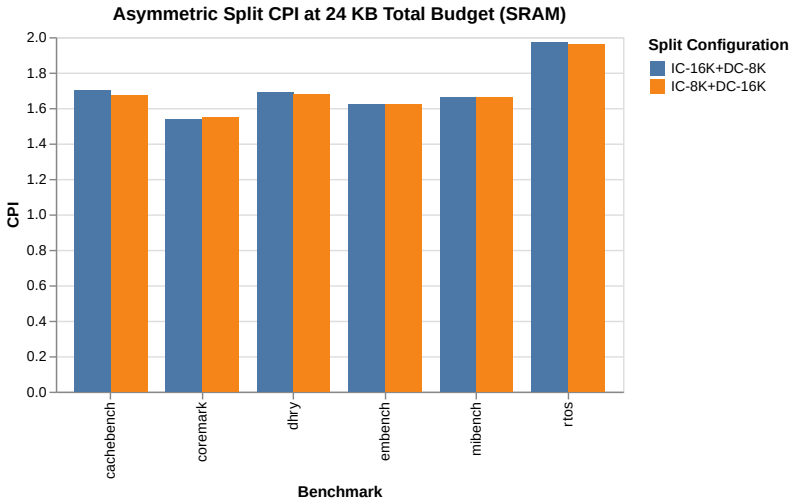
The two 12 KB splits are essentially equivalent on average — each wins on different benchmarks. IC-8K+DC-4K is preferable for `coremark` (IC-dominant) and `rtos` (lower CPI from bigger I-cache reducing kernel fetch stalls). IC-4K+DC-8K is better for `dhry`, `cachebench`, and `mibench`. The practical implication is:

- Either asymmetric 12 KB split approximately matches the symmetric IC-8K+DC-8K (16 KB total) for most benchmarks
- The symmetric IC-8K+DC-8K remains the safer choice because it avoids the workload-specific sensitivity of the split configurations

6.3. Asymmetric Split Comparison at 24 KB Total Budget

With a 24 KB total cache budget the two natural splits are IC-16K+DC-8K (I-cache dominant) and IC-8K+DC-16K (D-cache dominant).

Config (SRAM)	dhry	coremark	rtos	cachebench	embench	mibench
IC-16K+DC-8K	1.690	1.540	1.970	1.700	1.620	1.660
IC-8K+DC-16K	1.680	1.550	1.960	1.670	1.620	1.660



The two 24 KB splits produce near-identical average CPI. IC-8K+DC-16K is marginally better for `cachebench` (1.67 vs. 1.70) because the 16 KB dataset sweep exercises D-cache capacity directly. Both 24 KB configurations closely match the symmetric IC-16K+DC-16K (32 KB total), confirming that 24 KB is sufficient for this workload mix; the additional 8 KB costs less than 0.03 CPI improvement on average.

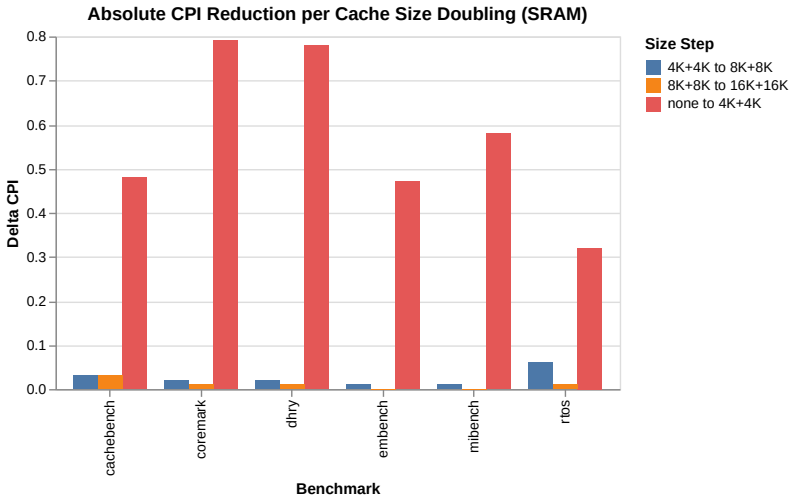
The same pattern that emerged at 12 KB repeats here: neither side dominates across all workloads, and balanced splits within a given total budget are the pragmatically safer choice unless the target workload has a strong I-cache or D-cache bias confirmed by profiling.

7. Marginal Gain: Doubling Cache Size

Understanding diminishing returns is critical for silicon area budgets. The following table shows the absolute CPI reduction from doubling each cache level.

7.1. CPI Delta from Doubling Cache (SRAM)

Size step	dhry	coremark	rtos	cachebench	embench	mibench
none to 4K+4K	0.780	0.790	0.320	0.480	0.470	0.580
4K+4K to 8K+8K	0.020	0.020	0.060	0.030	0.010	0.010
8K+8K to 16K+16K	0.010	0.010	0.010	0.030	0.000	0.000



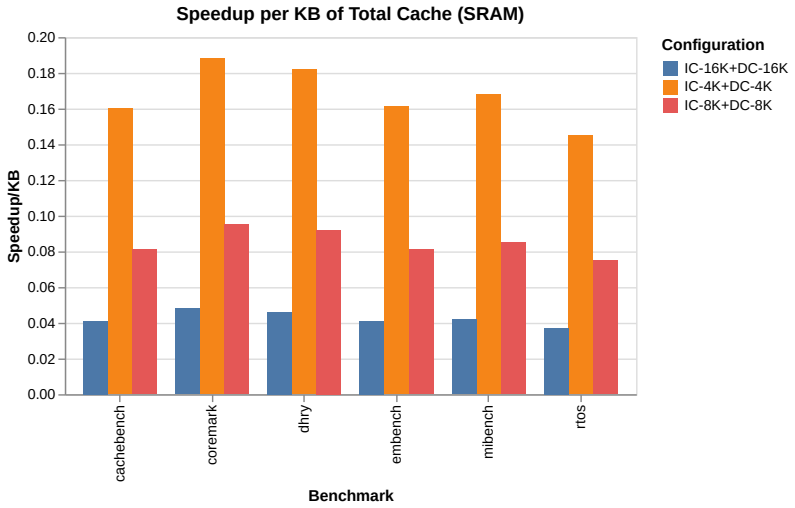
The jump from no cache to 4K+4K dominates by 10-80x compared to subsequent doublings. The 4K to 8K step is the smallest in absolute terms for most benchmarks. The 8K to 16K step is similarly modest — only `cachebench` and `rtos` show perceptible gains at 16K+16K because their working sets just exceed 8K capacity.

Silicon area implication: In a 28 nm process, 1 KB of 6T SRAM costs roughly 0.006 mm² (a rough estimate). The 4K+4K to 8K+8K step doubles the committed SRAM area (8 KB extra) while yielding only 0.01-0.06 CPI reduction. For most workloads, the 8K to 16K step similarly adds 16 KB extra SRAM for less than 2% performance gain. Whether this trade is worthwhile depends on the die area budget and the application's sensitivity to execution speed.

8. Cache Area Efficiency (Performance Gain per KB)

To compare configurations on a normalised basis, we compute speedup per KB of total cache: (Speedup over no-cache) / (total KB).

Config (SRAM)	dhry	coremark	rtos	cachebench	embench	mibench
IC-4K+DC-4K	0.182	0.188	0.145	0.160	0.161	0.168
IC-8K+DC-8K	0.092	0.095	0.075	0.081	0.081	0.085
IC-16K+DC-16K	0.046	0.048	0.037	0.041	0.041	0.042



The efficiency roughly halves with each doubling, confirming the diminishing-return law. IC-4K+DC-4K is the most area-efficient configuration by a significant margin. This is the "knee of the curve": for applications where die cost or power are constrained, IC-4K+DC-4K delivers the best performance per unit silicon area.

9. Associativity: 2-Way vs. 4-Way

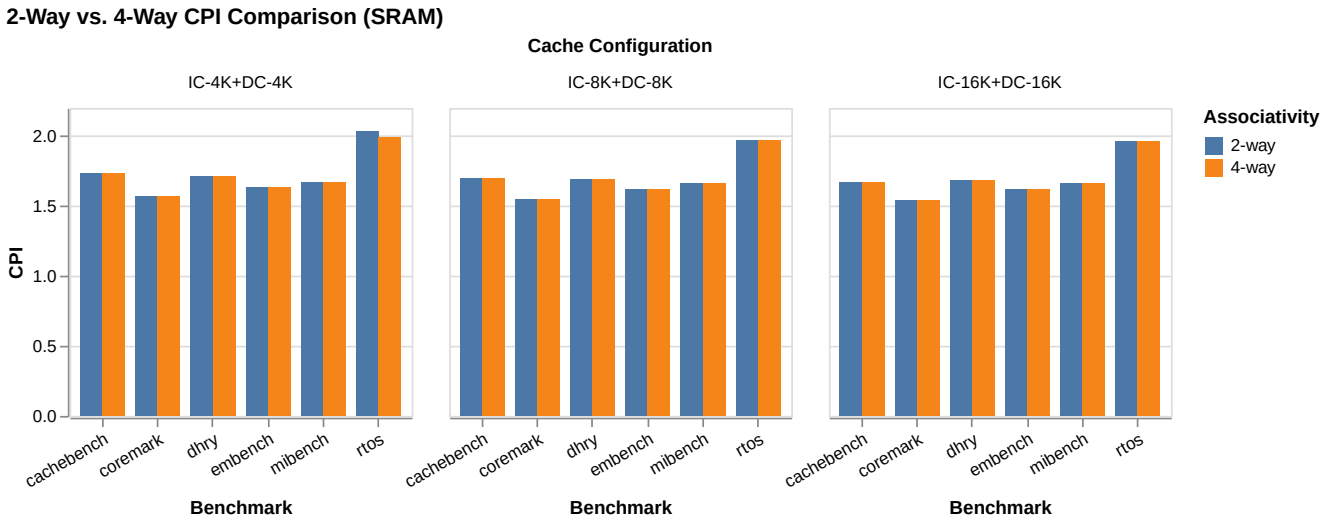
Set-associative caches use multiple ways to reduce **conflict misses** — the situation where two frequently accessed addresses map to the same cache set and repeatedly evict each other. Increasing from 2-way to 4-way doubles the number of lines per set, allowing more address mappings to coexist without conflict. The trade-off is additional tag-comparison logic and slightly longer hit latency.

This sweep added three 4-way configurations (IC-4K+DC-4K, IC-8K+DC-8K, IC-16K+DC-16K) to quantify the practical benefit of higher associativity on KV32 workloads.

9.1. CPI Comparison: 2-Way vs. 4-Way (SRAM)

Config (SRAM)	dhry 2W	dhry 4W	rtos 2W	rtos 4W	coremark 2W	coremark 4W	cachebench 2W	cachebench 4W
IC-4K+DC-4K	1.710	1.710	2.030	1.990	1.570	1.570	1.730	1.730
IC-8K+DC-8K	1.690	1.690	1.970	1.970	1.550	1.550	1.700	1.700
IC-16K+DC-16K	1.680	1.680	1.960	1.960	1.540	1.540	1.670	1.670

Config (SRAM)	embench 2W	embench 4W	mibench 2W	mibench 4W
IC-4K+DC-4K	1.630	1.630	1.670	1.670
IC-8K+DC-8K	1.620	1.620	1.660	1.660
IC-16K+DC-16K	1.620	1.620	1.660	1.660



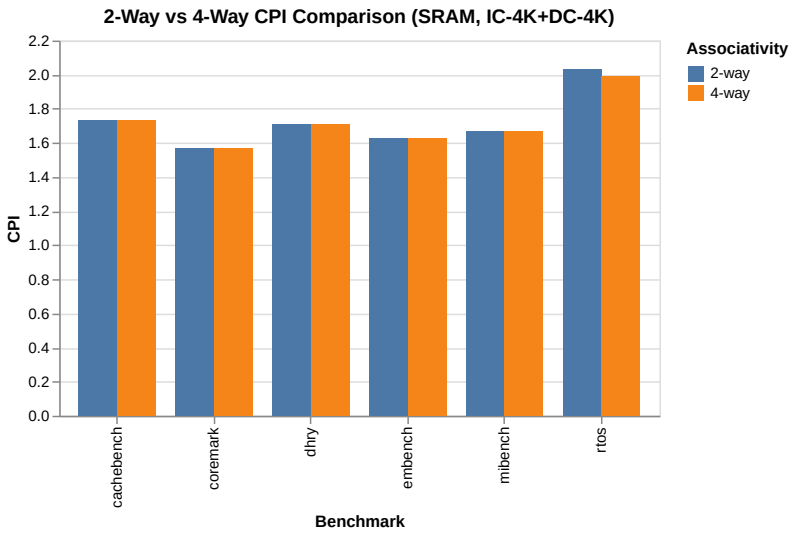
The 4-way configurations produce **essentially identical CPI** to 2-way across all benchmarks and sizes on SRAM. The single exception is `rtos` at IC-4K+DC-4K, where 4-way reduces CPI from 2.030 to 1.990 — a 2.0% improvement.

9.2. CPI Comparison: 2-Way vs. 4-Way (DDR4-1600)

The benefit of 4-way is slightly more visible on DDR4-1600, where cache misses cost more cycles:

Config (DDR4-1600)	dhry	coremark	rtos	cachebench	embench	mibench
IC-4K+DC-4K 2W	1.870	1.700	2.260	2.080	1.820	1.820
IC-4K+DC-4K 4W	1.870	1.700	2.140	2.080	1.840	1.810
IC-8K+DC-8K 2W	1.830	1.650	2.050	1.970	1.800	1.770
IC-8K+DC-8K 4W	1.830	1.650	2.040	1.970	1.800	1.770

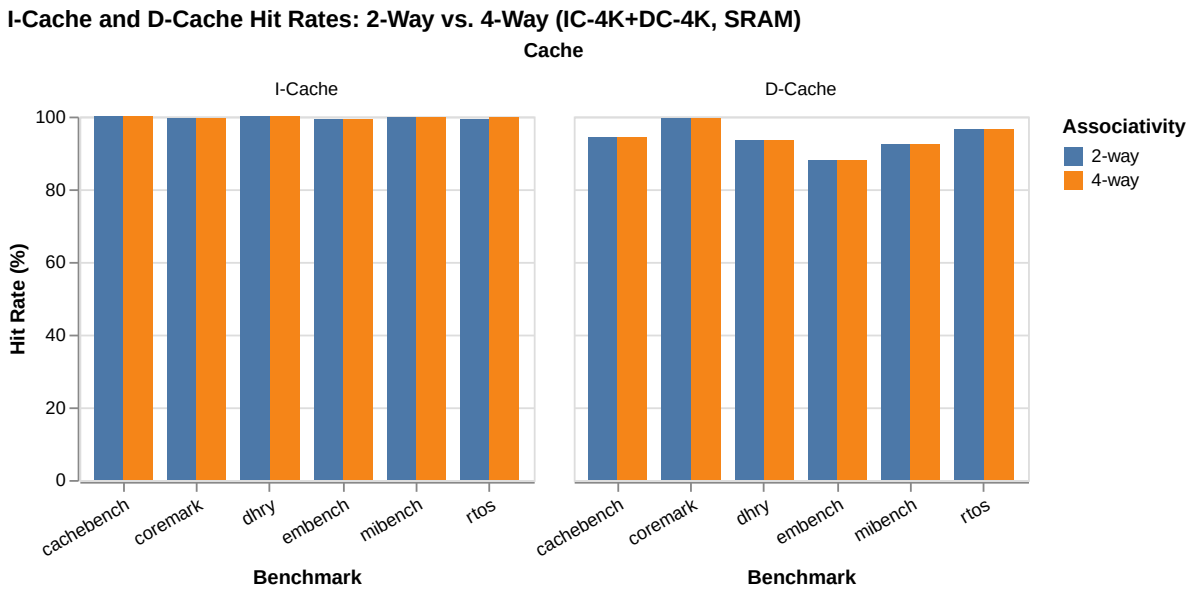
On DDR4-1600, `rtos` at IC-4K+DC-4K improves by 5.3% with 4-way (2.260 → 2.140). Other benchmarks remain unchanged. The signal is cleaner at higher miss penalty because each resolved conflict miss at DDR4 latency saves more cycles.



9.3. Hit Rate Analysis

Examining I-cache and D-cache hit rates confirms why the 4-way benefit is small:

Benchmark (IC-4K+DC-4K, SRAM)	IC hit 2W	IC hit 4W	DC hit 2W	DC hit 4W
dhry	99.88%	99.88%	93.46%	93.46%
coremark	99.55%	99.54%	99.49%	99.52%
rtos	99.26%	99.79%	96.36%	96.49%
cachebench	99.99%	99.99%	94.18%	94.16%
embench	99.27%	99.24%	88.03%	87.93%
mibench	99.77%	99.78%	92.38%	92.44%



The I-cache hit rates are already >=99.26% at 2-way. Conflict misses in a 2-way 4K I-cache occur when two hot loops map to the same set and thrash each other. For these benchmarks the hot code footprints are small enough that this rarely happens. The `rtos` entry is the exception: the Mini-RTOS

context switch dispatcher and the task bodies hash to overlapping sets. 4-way resolves this conflict, improving rtos I-cache hit rate from 99.26% to 99.79% (3x fewer misses) and accounting for the observed CPI improvement.

9.4. Why 4-Way Rarely Helps on SRAM

The miss penalty on SRAM is ~8 CPU cycles per refill (8-beat burst at 1-cycle AXI latency). With already high hit rates (99%+) and a low miss penalty, the formula:

```
stall_cycles = miss_count x miss_penalty
```

means that even eliminating half the remaining misses via 4-way associativity saves a negligible fraction of total stall cycles. At DDR4 latency (30-50 cycles per miss), the same improvement would save substantially more cycles — and indeed the DDR4 `rtos` result shows a 5.3% improvement vs. 2.0% on SRAM.

9.5. Recommendation: Associativity

- **2-way is sufficient for all benchmarks except rtos.** For general-purpose embedded workloads on SRAM, moving from 2-way to 4-way provides < 1% benefit at the cost of increased tag logic area and marginally longer hit path.
- **4-way is worth considering for RTOS-heavy workloads on DDR4.** If the system runs MiniRTOS on DDR4 and the thermal/area budget permits, 4-way at IC-4K+DC-4K saves 5% CPI on the task-switch microkernel workload.
- **Do not use 4-way to compensate for insufficient cache size.** If the working set exceeds the cache capacity, capacity misses — not conflict misses — are dominant. Doubling the cache size (e.g., 4K to 8K) eliminates far more misses than quadrupling the associativity.

10. Configuration Recommendations

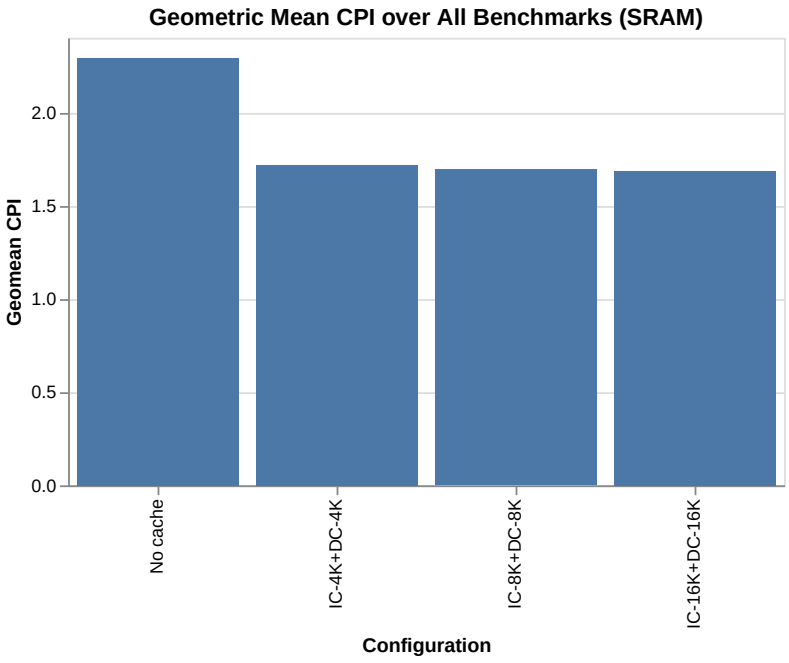
The following decision table maps deployment scenarios to recommended cache configurations based on workload profile, memory type, and silicon area constraints.

10.1. Decision Table

Use Case	Dominant Workload	Memory	Recommended Config	Rationale
Microcontroller (area-first)	Control loop, ISR	SRAM	IC-4K+DC-4K 2W	Best efficiency/KB; working set fits

Use Case	Dominant Workload	Memory	Recommended Config	Rationale
General embedded (balanced)	Mixed RTOS+compute	SRAM	IC-8K+DC-8K 2W	Sweet spot; 2x area for 2-3% extra gain
Performance embedded	All benchmarks	SRAM	IC-16K+DC-16K 2W	Highest perf; justify extra 16 KB SRAM
DDR4 system (cost-first)	Boot + application	DDR4	IC-8K+DC-8K 2W	Masks DDR latency; 10x speedup vs no-cache
DDR4 high-performance	Applications	DDR4	IC-16K+DC-16K 2W	Residual DDR penalty < 10% at 16K+16K
RTOS on DDR4	Mini-RTOS tasks	DDR4	IC-4K+DC-4K 4W	4-way saves 5% on context-switch bound code
DSP / data streaming	Large filter ops	Any	IC-4K+DC-16K 2W	Data-dominant; maximise D-cache share

10.2. Average CPI by Config (SRAM, 6-Benchmark Geomean)



The geometric mean is computed as the nth root of the product of all benchmark CPIs. IC-4K+DC-4K captures the vast majority of available performance (geomean drops to 1.717 from 2.290 uncached),

while 8K+8K and 16K+16K provide progressively smaller but still meaningful gains for performance-critical deployments.

10.3. What NOT to Use

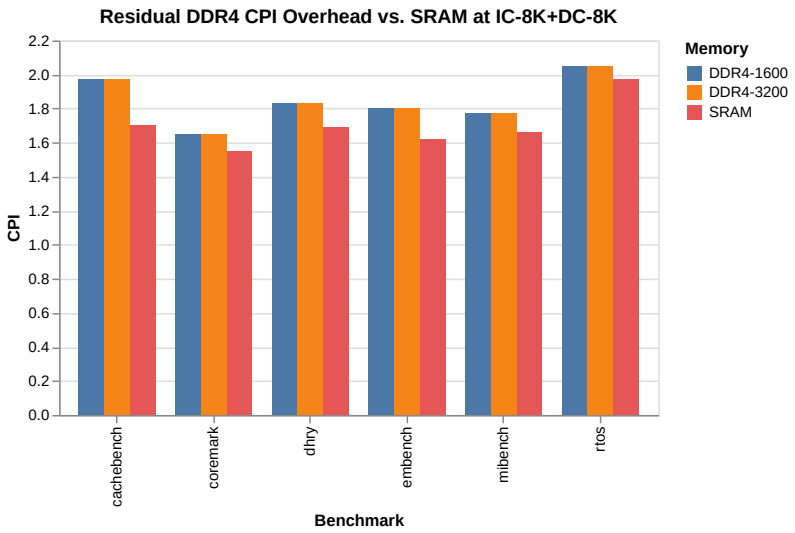
Anti-Pattern	Reason to Avoid
IC-4K + DC-off on DDR4	Instruction bursts are cached but every load/store hits DDR4 with full CAS latency. CPI = 10-17x uncached.
IC-off + DC-16K on any memory	55-65% of all stall cycles on a typical workload are instruction-fetch stalls. A D-cache alone cannot help those cycles.
IC-16K+DC-16K on SRAM for cost-sensitive	32 KB of SRAM with < 1% CPI improvement over 16 KB (8K+8K). Only worth it if the workload is known to have a >8 KB data footprint.
4-way instead of bigger cache	If working set exceeds cache capacity, capacity misses dominate. Doubling cache size eliminates far more misses than 4-way associativity.

11. DDR4 vs. SRAM: Detailed Comparison

11.1. Residual CPI Penalty on DDR4 with Full IC-8K+DC-8K

Even with IC-8K+DC-8K, there is a residual DDR4 penalty from the few accesses that miss the cache. The table below shows the CPI overhead of DDR4 relative to SRAM for the same IC-8K+DC-8K configuration.

Memory	dhry	coremark	rtos	cachebench	embench	mibench
SRAM	1.690	1.550	1.970	1.700	1.620	1.660
DDR4-1600	1.830	1.650	2.050	1.970	1.800	1.770
DDR4-3200	1.830	1.650	2.050	1.970	1.800	1.770
DDR4-1600 overhead	+8.3%	+6.5%	+4.1%	+15.9%	+11.1%	+6.6%
DDR4-3200 overhead	+8.3%	+6.5%	+4.1%	+15.9%	+11.1%	+6.6%



The residual DDR4 penalty at IC-8K+DC-8K ranges from 4-16%, with `cachebench` being the outlier at 15.9%. This reflects the fact that `cachebench's 16 KB sweeping dataset exceeds the D-cache capacity and causes sustained cache misses that reach DRAM.

DDR4-1600 vs. DDR4-3200: In this cache-dominated regime, both DDR4 speeds produce identical CPI at 2-decimal resolution. The residual misses reach DRAM infrequently enough that the bandwidth difference between 1600 and 3200 mt/s creates no measurable benefit. System designers can choose DDR4-1600 without a performance penalty for these workloads when caches are sized at IC-8K+DC-8K or larger.

12. Conclusions

1. **Caches are mandatory for DDR4 systems.** Without caches, DDR4 CPI is 15-19x. Even IC-4K+DC-4K brings DDR4 performance within 35-50% of SRAM performance; IC-8K+DC-8K achieves within 5-16%.
2. **IC-4K+DC-4K is the performance knee.** The first 8 KB of cache (4K+4K) eliminates 14-33% of the no-cache CPI on SRAM and is by far the most area-efficient configuration. This is the minimum recommended configuration for any real deployment.
3. **IC-8K+DC-8K is the recommended default.** At 16 KB total, it provides 1-3% additional improvement over 4K+4K with well-characterised behaviour across all workloads. It balances performance, area, and generality.
4. **IC-16K+DC-16K offers marginal gains over 8K+8K.** The 8K to 16K doubling yields less than 2% CPI improvement for most benchmarks. It is only justified when `cachebench`-like bandwidth workloads or large RTOS task sets are representative of production use.
5. **Asymmetric splits provide no clear winner.** IC-8K+DC-4K and IC-4K+DC-8K have essentially equal average CPI for these workloads. Both are slightly worse in average CPI than the symmetric IC-8K+DC-8K. The symmetric configuration is recommended unless the workload is known to be strongly I-cache or D-cache bound.
6. **4-way associativity is rarely beneficial on SRAM.** With hit rates already at 99%+ and an 8-cycle SRAM miss penalty, eliminating conflict misses via 4-way saves < 2% CPI across all benchmarks. The one exception is `rtos` at IC-4K+DC-4K, where 4-way resolves context-switch code conflicts for a 2% improvement. On DDR4, the `rtos` improvement reaches 5.3%, making 4-

way worth considering for RTOS-heavy DDR4 systems.

7. **Stall cycles never reach zero.** Even at IC-16K+DC-16K, 35-49% stall fraction remains due to compulsory misses, AXI bus pipeline overhead, and structural hazards. Further reductions would require hardware prefetching, non-blocking cache fills, or loop-body partitioning — all beyond the scope of this analysis.

Appendix A: Test Methodology

A.1. Simulation Environment

All measurements were obtained via RTL simulation of the full KV32 SoC, including the AXI crossbar, CLINT, and memory models.

- **Simulator:** Verilator (cycle-accurate RTL simulation)
- **Memory model (SRAM):** `mem_axi.sv` with 1-cycle AXI response
- **Memory model (DDR4):** `ddr4_axi4_slave.sv` — parameterised DDR4 timing model with configurable CAS latency, tRCD, and tRP
- **AXI bus width:** 32-bit data width; 8-beat bursts for cache line fills (32 bytes per line)

A.2. Metric Accounting

All metrics are computed over the full benchmark run, from first instruction fetch to the final `ecall` that terminates the benchmark.

- **CPI:** reported directly by the KV32 simulation statistics output
- **Stall %:** `stall_cycles / total_cycles * 100`
- **I-Cache hit rate:** `(I_accesses - I_misses) / I_accesses`
- **D-Cache hit rate:** `(D_accesses - D_misses) / D_accesses`

Retired instructions are counted at WB stage, not Fetch stage. Flushed instructions (due to branch mispredictions or exception handling) are not counted as retired.

A.3. Cache Parameters

The KV32 I-cache and D-cache share the same microarchitecture:

- **Organisation:** 2-way or 4-way set-associative (configurable)
- **Line size:** 32 bytes (8 words)
- **Replacement policy:** LRU (least-recently used)
- **D-cache write policy:** write-back with write-allocate
- **D-cache coherence:** single-core; no hardware coherence protocol

A.4. Benchmark Compilation

All benchmarks were compiled with RISC-V GCC (riscv32-unknown-elf-gcc) targeting RV32IMAC with `-O2` optimisation. The same binary was used for all cache configurations; no benchmark-specific tuning was applied.

Appendix B: Build Instructions

B.1. Required Tools

```
gem install asciidoctor-pdf asciidoctor-diagram
npm install -g vega-lite vega-cli
```

B.2. Generating the PDF

From the `docs/` directory:

```
asciidoctor-pdf -r asciidoctor-diagram cache_benchmark_report.adoc
```

The generated file is `cache_benchmark_report.pdf`. Chart images are cached in `docs/images/` and can be deleted once the PDF is built.

B.3. Raw Data

Benchmark results are in `build/cache_bench_v2/results.csv` (234 runs, all PASS).